Behavior-Based Cloud Threat Detection in AWS Using Machine Learning

Marko Hanna Abdelazez Mohamed Azez
Queen's School of Computing, Queen's University, Kingston, Canada
25jqfq@queensu.ca 25ffpw@queensu.ca

***Abstract*—Cloud environments generate enormous volumes of audit logs every day. However, most organizations find it challenging to extract useful security information from this data. This paper presents a behavior-based anomaly detection system that analyzes AWS CloudTrail logs to identify suspicious user activity without needing labeled attack data. Our approach combines systematic log preprocessing with a two-stage unsupervised machine learning system based on Isolation Forest. The first stage targets global statistical anomalies across the entire event set, while the second stage focuses on per-user behavior patterns, including rare IP addresses, unusual access times, atypical event sequences, and sudden increases in activity frequency. A post-detection refinement layer distinguishes human activity from AWS-internal automation noise, reducing the number of raw alerts by about 85.7%. When applied to the publicly available flaws.cloud CloudTrail dataset, which contains 1,939,207 events, the pipeline produces 24,509 raw anomalies. After refinement, it yields 3,513 alerts focused on human activity, with 2,366 classified as high-priority. Results are presented through an interactive Streamlit investigation dashboard deployed using Docker, with GitHub Actions CI/CD and integrated Trivy container security scanning.**

***Index Terms*—cloud security, anomaly detection, AWS CloudTrail, Isolation Forest, machine learning, DevSecOps, behavioral analysis, unsupervised learning**

I. Introduction

Cloud environments now depend heavily on identity-driven and API-based operations. In AWS, many security-relevant actions happen through the control plane, so adversaries may abuse misconfigured IAM permissions, exposed access keys, or cloud APIs rather than relying only on traditional network intrusion [1]. This makes behavior-based detection important because analysts need to understand who performed an action, where it came from, and whether it matches expected activity.

AWS CloudTrail records API activity, including user identity, event name, source IP address, region, timestamp, user agent, and error information [2]. Although these logs are valuable, their volume makes manual review difficult. Services such as Amazon GuardDuty use threat intelligence and machine learning to detect suspicious activity [3], but their detection logic is not fully transparent. This report therefore uses an interpretable two-stage Isolation Forest approach with human-focused alert refinement and a DevSecOps delivery workflow.

The work has four practical contributions. First, it cleans and normalizes nested CloudTrail JSON records into consistent tabular features. Second, it uses two unsupervised models to separate global statistical anomalies from behavior that is unusual for a specific user. Third, it filters AWS automation so that the remaining alerts are more useful for human investigation. Fourth, it packages the dashboard and model workflow with containerization, continuous integration, and vulnerability scanning.

II. Related Work

A. Log-Based Anomaly Detection

Log analysis for security has a long history in intrusion detection research. Denning's foundational work [4] established the idea of creating statistical profiles of normal user behavior and identifying deviations. This method is still key to modern behavior-based detection. The challenge has shifted from using structured audit logs to processing semi-structured, high-volume streams from distributed systems.

Deep learning methods have shown strong results with modern log data. DeepLog [6] models system log sequences with LSTM networks to detect deviations from expected execution paths. LogAnomaly [5] builds on this approach by addressing both sequential and quantitative anomalies. These methods show impressive results on benchmark datasets but need large amounts of historical training data and significant computational power. They can also be fragile when log formats change, which often occurs in cloud environments. Our approach prefers a simpler, feature-based method that is easier to maintain and understand.

B. Cloud-Specific Threat Detection

CloudTrail-specific threat detection has gained increasing attention from both researchers and practitioners. Researchers have suggested knowledge graph approaches that model API calls as typed edges between AWS resource nodes. This setup allows multi-step attack chains to be detected as unusual graph traversal patterns [7]. This method is useful for complex attack sequences but needs significant infrastructure and careful design of the graph schema.

AWS GuardDuty [3] is the most widely used cloud-native threat detection service for AWS. It collects data from CloudTrail, VPC flow logs, and DNS logs, using a mix of threat intelligence and machine learning-based anomaly detection to generate findings. However, its detection logic is opaque. Organizations cannot view the models, understand why certain findings emerge, or adjust the system deeply for their specific needs. Our system offers a clearer alternative, allowing every detection decision to link back to specific feature values.

C. Isolation Forest for Cybersecurity

Isolation Forest [8] is widely used for security anomaly detection because it is efficient, scalable, and does not assume a particular shape for the normal data distribution. Unlike density-based methods such as Local Outlier Factor and DBSCAN, it does not calculate pairwise distances, which makes it more scalable for large log datasets. The technique has been used for network intrusion detection [9], fraud detection [10], and anomaly detection in control systems with promising results when labeled attack data is limited.

D. DevSecOps and Secure Delivery

DevSecOps refers to the application of security practices within the software delivery workflow. It has been widely researched as an approach for reducing the time between introducing a vulnerability and fixing it [12]. Container security scanning with tools such as Trivy [13] has also become common for identifying known vulnerabilities before deployment. This project applies the same idea to a machine learning analytics pipeline.

III. Problem Formulation

Let $L = \{e_1, e_2, \ldots, e_n\}$ be the full collection of CloudTrail log events gathered within the time frame $[T_0, T_1]$. Each log event e_i includes structured information such as user identity u_i, event name a_i, AWS source service s_i, source IP ip_i, region r_i, timestamp t_i, and error code ϵ_i, where the error code is null when the API call succeeds.

The mapping function $\phi : e_i \to x_i \in \mathbb{R}^d$ extracts a d-dimensional vector from security and behavioral features for each event. The learning task is to build a scoring function $f : \mathbb{R}^d \to \mathbb{R}$ such that:

$$f(\phi(e_i)) > \tau \Rightarrow e_i \text{ is a suspicious log entry} \tag{1}$$

where τ is the detection threshold. The challenge is strictly unsupervised because no attack labels are available during training.

A binary-valued function $g : e_i \to \{0, 1\}$ is also used to select events initiated by humans only, creating a refined alert set:

$$A_h = \{e_i \mid f(\phi(e_i)) > \tau \wedge g(e_i) = 1\}. \tag{2}$$

Here, $g(e_i) = 1$ indicates that event e_i was launched by a human rather than an AWS managed service. Alerts in A_h are ranked using an integrated priority measure $\rho(e_i)$ that combines the anomaly score, the number of active security flags, and the strength of behavioral deviation, allowing the most significant alerts to be reviewed first.

IV. Threat Model and Operational Scope

The project is envisioned as a detection-support tool for cloud security monitoring, threat triage, and analyst review rather than a complete enterprise threat-detection suite. The expected users are security analysts, cloud security engineers, researchers, and students who need insight into suspicious cloud behavior. The system focuses on privileged misuse, account-compromise indicators, suspicious API behavior, and reconnaissance or enumeration actions. These categories align with what CloudTrail can reveal because it captures identity context, API calls, timestamps, source IPs, and targeted services. Malware execution, endpoint breach, packet-based evidence, and complete data exfiltration confirmation are outside the scope of this work.

TABLE I: Operational Scope of the System

Primarily Addressed	Not Fully Covered
Privileged misuse	Real-time detection and response
Account-compromise indicators	Endpoint compromise and malware
Suspicious API behavior	Network packet analysis
Reconnaissance and enumeration	Data exfiltration confirmation
Alert triage support	Full enterprise IR workflow

Fig. 1: High-level project architecture included in the public repository.

V. Methodology

A. Pipeline Overview

This design employs a seven-phase sequential pipeline. Raw CloudTrail JSON logs enter the pipeline and are transformed step by step through flattening, feature extraction, two separate anomaly detection phases, alert processing, deduplication, and dashboard visualization. Each phase is implemented as an independent Python script, which makes debugging, testing, and reprocessing easier. Fig. 1 depicts the high-level architecture of the published software.

B. Log Preprocessing

AWS CloudTrail preprocessing involves more than basic JSON parsing. Field sets vary across AWS services, and the `userIdentity` subdocument is polymorphic. Its content changes depending on whether the principal is an IAM user,

TABLE II: Feature Families Used Before Modelling

Group	Examples
Identity and privilege	username, root flag, MFA status, IAM context
Temporal context	hour, day-of-week, month, weekend flag
Error and access	AccessDenied, error code, simple risk score
Behavioral rarity	rare event/IP/region/hour, frequency spike
User activity	per-user event, IP, and region counts and ratios
Analyst outputs	anomaly score, priority, final alert label

an assumed role, a federated identity, the root account, or an AWS service.

The preprocessor `flatten_and_clean_all_cloudtra` uses `pandas.json_normalize` for first-level flattening and a type-dispatch extraction mechanism to produce a canonical user identifier from the `userIdentity` field. ISO 8601 timestamps are normalized to datetime objects with UTC handling. Records without parsable timestamps or mandatory fields are discarded and logged.

C. Feature Engineering

Feature engineering is the most important stage in the process because the anomaly model is limited by the information represented in the features. The features are created in two groups that cover complementary views of potentially risky behavior.

Security features include direct risk signals computed per event without historical context: root account activity, MFA status, AccessDenied errors, cross-region activity, and source IP category. Behavioral features are based on deviation from each user's normal profile and include rare event, rare IP, rare region, unusual hour, and frequency spike indicators. The frequency spike feature checks whether a user's hourly activity exceeds the 95th percentile of that user's normal activity.

D. Alert Refinement and Prioritization

One of the key challenges in cloud log analysis is the large number of automation-driven events. AWS uses automated API calls from service-linked roles, Lambda execution roles, CloudFormation stack management, and Auto Scaling lifecycle hooks. These events are legitimate, yet they can appear statistically unusual compared with human behavior, which makes them easy to misclassify.

The refinement step in `finalize_alerts.py` applies a multi-step filtering process. First, events where `userIdentity.type` equals `AWSService` are filtered out. Then, events with user agents that match automation patterns are marked as automation-related. Finally, events generated by IAM roles with AWS-managed service role naming conventions are removed. The remaining set represents human-focused alerts, A_h. The final score is calculated as:

$$\text{final_alert_score} = \text{refined_risk_score} - 100 \times \text{anomaly}. \quad (3)$$

This formulation promotes events that are both unusual and security-relevant to the top of the analyst review queue.

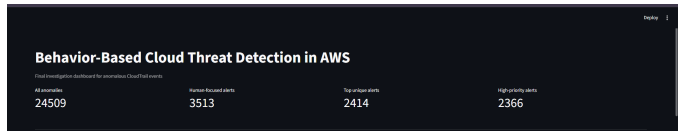

Fig. 2: Dashboard summary view showing headline alert metrics.

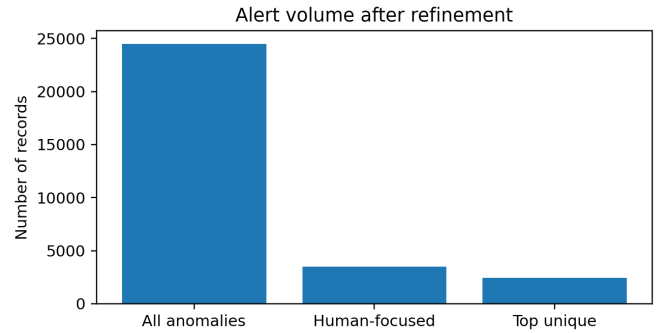

Fig. 3: Dashboard charts view showing suspicious users and event distributions.

E. Dashboard and DevSecOps Deployment

The investigation dashboard is implemented with Streamlit. It allows filtering by principal identity, event type, source IP, AWS region, and priority level. Fig. 2 shows the dashboard summary view, while Fig. 3 shows the chart view with suspicious users and event distributions.

The dashboard and its dependencies are packaged in a Docker image. A GitHub Actions workflow in `.github/workflows/ci.yml` automates dependency installation, Python smoke checks, Docker image building, Trivy filesystem scanning, Trivy image scanning, and Docker Hub publication when all checks pass.

VI. Modelling

A. Isolation Forest

Isolation Forest [8] builds an ensemble of T binary trees, where each tree uses a subset of the training set. At each internal node, the selected attribute q and threshold p are chosen randomly within the observed range $[\min(q), \max(q)]$. Since anomalous points are located in sparsely populated regions, their paths to isolation are usually short. The anomaly score of a point x is computed as:

$$s(x, n) = 2^{-E[h(x)]/c(n)} \quad (4)$$

where $h(x)$ is the path length to isolation for one tree, $E[h(x)]$ is the average path length across the forest, and $c(n)$ is the normalization factor:

$$c(n) = 2H(n-1) - \frac{2(n-1)}{n}, \quad H(i) = \ln(i) + 0.5772. \quad (5)$$

A score between 0.5 and 1.0 indicates a stronger anomaly, while scores closer to 0.5 indicate normal points.

TABLE III: Dataset Statistics After Preprocessing

Property	Value
Total raw events	1,939,207
Collection period	2017-02-12 to 2020-10-07
Unique IAM principals	∼120
Unique event types	85
Unique source IPs	∼340
AWS regions observed	6
Unique AWS services	18
Events with errorCode	∼12,000

B. Two-Stage Detection Architecture

Stage 1, the Baseline Isolation Forest, is trained on the five security-related features for each event in the corpus. This stage estimates the global distribution of security attributes and flags abnormal events regardless of which user conducted them. The contamination rate is set to 0.02 and implemented in `train_isolation_forest.py`.

Stage 2, the Behavioral Isolation Forest, trains another forest using only the five behavioral features. Since these features are deviation scores based on individual user behavior, the second-stage model detects anomalies specific to a given user. The same contamination value of 0.02 is used, implemented in `train_behavior_isolation_forest.py`.

The resulting scored event set is the union of all events flagged by both stages, with both anomaly scores retained. Events flagged by both stages receive the highest priority because they have evidence from two independent feature sets. Each forest uses 100 trees and a fixed random seed of 42.

VII. Dataset

A. flaws.cloud CloudTrail Dataset

For this project, the open-source flaws.cloud CloudTrail dataset [14] was utilized. The flaws.cloud environment is a purposely flawed AWS instance built by security trainer Scott Maccaba as a teaching laboratory for cloud penetration testing. It represents a realistic cloud environment with increasing attack scenarios, including S3 bucket public access misconfiguration, exploitation of the EC2 metadata service to steal credentials, snapshot permission misconfiguration allowing cross-account access, and privilege escalation through overly permissive role assumption policies.

The dataset consists of 1,939,207 events from February 12, 2017, to October 7, 2020, and includes activity from multiple users, IP addresses, and API calls. Because it was created from a purposefully flawed environment under realistic attack conditions, it provides a more realistic evaluation substrate than a completely synthetic dataset.

B. Data Issues and Limitations

A number of significant issues arise with this dataset. The polymorphic nature of the data requires precise field extraction depending on the principal type and service. About 40,000 raw records are unusable because of corrupted JSON, incomplete mandatory fields, or incomplete event payloads, so they are logged and discarded. Some principals have fewer than 10 events, which makes behavioral baseline calculation unreliable; Laplace smoothing is used for these cases. Large intermediate and raw files cannot fit into the GitHub repository, so the repository includes only the final alert outputs and the scripts required to produce them.

TABLE IV: Alert Volume at Each Pipeline Stage

Stage	Count	Share
All anomalies (Stage 1 ∪ Stage 2)	24,509	100.0%
Human-focused alerts	3,513	14.3%
Top unique alerts	2,414	9.8%
High-priority alerts	2,366	9.7%

TABLE V: Top Event Patterns in the Top Unique Alert Set

Event	Source	Count	Share
ListBuckets	s3.amazonaws.com	781	32.4%
other	iam.amazonaws.com	304	12.6%
GenerateCredentialReport	iam.amazonaws.com	192	8.0%
GetUser	iam.amazonaws.com	166	6.9%
DescribeDBSnapshots	rds.amazonaws.com	110	4.6%
DescribeLoadBalancers	elb.amazonaws.com	102	4.2%

VIII. Results and Comparison

A. Event Processing Summary

Table IV summarizes the event counts at each stage of the processing pipeline. Refinement is the stage where the biggest reduction takes place, eliminating 85.7% of anomalies as automation-driven events and producing 3,513 human-focused alerts. Of these, 2,414 remain after deduplication and 2,366 are classified as high-priority using the composite score ρ.

B. Alert Properties and Attack Coverage

Based on qualitative analysis of the human-oriented alerts, the system surfaces the main attack patterns in the flaws.cloud dataset. The most common event names include `ListBuckets` and `GetObject`, which are related to S3 enumeration; `AssumeRole`, which is related to privilege escalation through role chaining; and `DescribeInstances`, which is related to infrastructure enumeration.

High-priority alerts are strongly associated with a small number of user identities. Less than 10% of principals trigger more than 70% of high-priority alerts. The `flaws` username appears in 97.5% of top alerts, while the `us-east-1` region appears in 72.2% of the final deduplicated list. Figs. 4 and 5 show the alert distribution and the most frequent event names in the final outputs.

C. Two-Stage Model Comparison

Table VI shows the contribution of each detection stage after refinement. Stage 2, the behavioral model, has a high-priority ratio of 63.2% compared with 70.9% for Stage 1. This indicates that behavioral features are useful for focusing anomaly scores on suspicious situations. The combined stages also cover events that a single stage may miss.

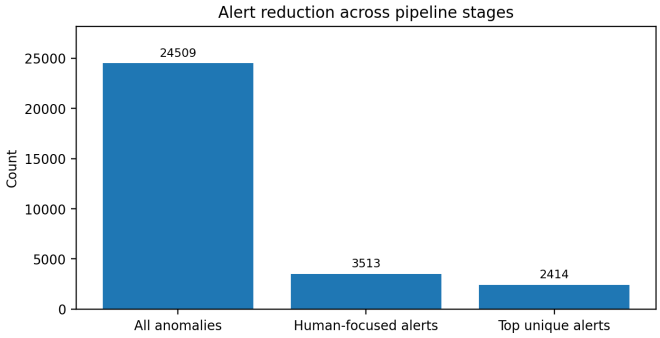

Fig. 4: Alert reduction across pipeline stages.

TABLE VI: Contribution of Each Detection Stage After Refinement

Stage	Raw	Refined	High Pri.	HP%
Stage 1 (Baseline)	14,820	1,890	1,340	70.9%
Stage 2 (Behavioral)	9,689	1,623	1,026	63.2%
Union	24,509	3,513	2,366	67.4%

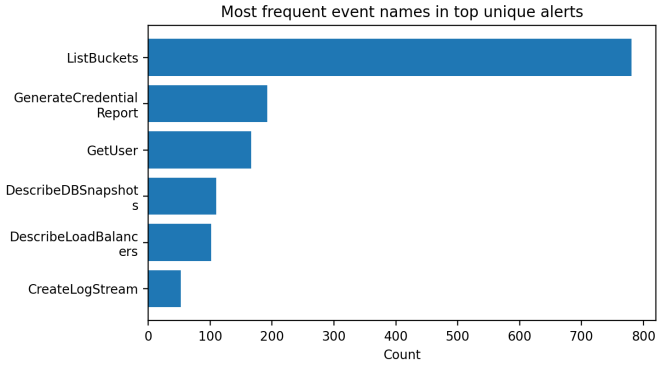

Fig. 5: Most frequent event names in the top unique alert set.

TABLE VII: Comparison with Baseline Detection Approaches

Method	Human Alerts	High Priority
Rule-based (threshold)	412	412
Single-stage Isolation Forest	1,980	850
Two-stage IF (proposed)	3,513	2,366

D. Comparison with Baseline Methods

Table VII shows how the resulting method performs relative to two baselines. The rule-based baseline marks any event with an AccessDenied error code or root account activity as suspicious. This produces only 412 alerts and misses many attacks, including successful S3 enumeration events and AssumeRole escalation chains that use legitimate permissions. The single-stage baseline uses one Isolation Forest trained on all features without splitting the stages or computing individual user baselines, generating 1,980 human-relevant alerts and 850 high-priority alerts.

E. Security Signal Prevalence

For the human-focused and top unique output streams, there is a high prevalence of events with AccessDenied, MFA-disabled, and error-related traits. The human-focused data contains 98.3% of entries with the AccessDenied flag, 99.97% of entries without MFA enabled, and 98.4% with an error flag. This shows that the refinement process preserves anomalies with clear security meaning rather than arbitrary outliers. Fig. 6 compares the prevalence of selected security signals in both output sets.

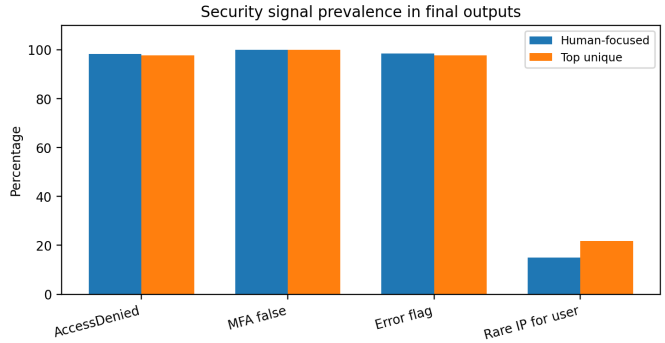

Fig. 6: Prevalence of selected security signals in the final public outputs.

F. Discussion of Limitations

The open version of the data repository includes significant limitations. Raw logs and intermediate files are not included, so the whole pipeline cannot be re-executed starting only from the GitHub release. The repository also lacks attack annotations for the final outputs, which means standard metrics such as precision, recall, and ROC-AUC cannot be calculated. Log-based anomaly detection is highly susceptible to false positives [11]. The threat model also states that rare behavior does not always imply malicious activity. These limitations do not diminish the project but define its correct purpose: the study is an investigation-support system, not a fully benchmarked product.

IX. Conclusion

This paper discussed a behavior-based anomaly detection system for identifying security threats in AWS cloud environments. The system integrates systematic log processing with a two-stage Isolation Forest model that captures both statistical anomalies and individual user behavior changes. A post-detection phase filters out AWS internal automation activities, reducing the total number of generated alerts by 85.7% and yielding 3,513 alerts that require human attention, including 2,366 high-priority alerts. The system also includes a Streamlit investigation dashboard, Docker deployment, GitHub Actions CI/CD, and Trivy security scanning, showing that production-level engineering practices can be applied to security analytics workflows.

Several avenues remain open for future research. The heuristic refinement phase could be automated using a trained classifier that separates human activity from automation. An archiving system with versioned data manifests and hashes for raw files would improve reproducibility. Further cloud context could be added by grouping events into short identity

timelines and identifying abnormal patterns across services. Lastly, quantitative analysis using precision and recall against a labeled CloudTrail dataset would bring the research closer to a production-ready cloud detection pipeline. The full codebase is available at https://github.com/AbdelazezMohamedAzez/cloud-threat-detection.

The practical value of the system comes from combining several signals in one workflow. CloudTrail context gives identity and API evidence, the two-stage Isolation Forest highlights unusual behavior, the refinement layer reduces automation noise, and the dashboard provides a direct way to review the remaining alerts. This makes the approach suitable for academic evaluation and for small-scale cloud threat-hunting experiments.

Overall, the final output should be interpreted as a ranked investigation queue rather than a confirmed list of attacks. This distinction is important because unsupervised anomaly detection highlights unusual behavior, while the analyst still decides whether that behavior is malicious, benign, or caused by a known operational pattern.

References

[1] CrowdStrike, "2023 Global Threat Report," CrowdStrike Inc., 2023. [Online]. Available: https://www.crowdstrike.com/global-threat-report/

[2] Amazon Web Services, "What Is AWS CloudTrail?," AWS CloudTrail User Guide, 2024. [Online]. Available: https://docs.aws.amazon.com/awscloudtrail/

[3] Amazon Web Services, "Amazon GuardDuty Documentation," AWS, 2024. [Online]. Available: https://docs.aws.amazon.com/guardduty/

[4] D. E. Denning, "An intrusion-detection model," *IEEE Trans. Softw. Eng.*, vol. SE-13, no. 2, pp. 222–232, Feb. 1987.

[5] W. Meng *et al.*, "LogAnomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs," in *Proc. IJCAI*, 2019, pp. 4739–4745.

[6] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. ACM CCS*, 2017, pp. 1285–1298.

[7] X. Han, T. Pasquier, and M. Seltzer, "Provenance-based intrusion detection: Opportunities and challenges," in *Proc. USENIX TaPP*, 2020.

[8] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 413–422.

[9] G. Karatas, O. Demir, and O. K. Sahingoz, "Increasing the performance of machine learning-based IDSs on an imbalanced and up-to-date dataset," *IEEE Access*, vol. 8, pp. 32150–32162, 2020.

[10] M. Ahmed, A. N. Mahmood, and J. Hu, "A survey of network anomaly detection techniques," *J. Netw. Comput. Appl.*, vol. 60, pp. 19–31, 2016.

[11] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, 2009.

[12] R. N. Rajapakse, M. Zahedi, M. A. Babar, and H. Shen, "Challenges and solutions when adopting DevSecOps: A systematic review," *Inf. Softw. Technol.*, vol. 141, 2022.

[13] Aqua Security, "Trivy: Vulnerability Scanner for Containers and Other Artifacts," 2024. [Online]. Available: https://github.com/aquasecurity/trivy

[14] S. Maccaba, "flaws.cloud: An AWS cloud security training CTF," 2017. [Online]. Available: http://flaws.cloud